

PRACTICAL 5:

Producer_consumer.c

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <unistd.h>
```

```
#include <string.h>
```

```
#include <sys/wait.h>
```

```
#include <time.h>
```

```
#define NUM_MESSAGES 1000
```

```
#define MESSAGE_SIZE 100
```

```
int main()
```

```
{
```

```
    int fd[2];
```

```
    pid_t pid;
```

```
    char message[MESSAGE_SIZE];
```

```
    clock_t start, end;
```

```
    double time_taken;
```

```
    if (pipe(fd) == -1)
```

```
    {
```

```
        perror("pipe failed");
```

```
        return 1;
```

```
    }
```

```
    start = clock();
```

```
pid = fork();

if (pid < 0)
{
    perror("fork failed");
    return 1;
}

if (pid == 0)
{
    // Child process = Consumer
    close(fd[1]);

    for (int i = 0; i < NUM_MESSAGES; i++)
    {
        read(fd[0], message, MESSAGE_SIZE);
    }

    close(fd[0]);

    printf("Child: Consumed %d messages\n", NUM_MESSAGES);
}
else
{
    // Parent process = Producer
    close(fd[0]);
```

```
for (int i = 0; i < NUM_MESSAGES; i++)
{
    snprintf(message, MESSAGE_SIZE,
        "Message %d from parent", i + 1);

    write(fd[1], message, MESSAGE_SIZE);
}

close(fd[1]);

wait(NULL);

end = clock();

time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;

printf("Parent: Produced %d messages\n", NUM_MESSAGES);
printf("Communication time: %f seconds\n", time_taken);

if (time_taken > 0)
{
    printf("Efficiency: %.2f messages/second\n",
        NUM_MESSAGES / time_taken);
}
}

return 0;
}
```

```
tarun@LAPTOP-BRD2JBFC:~/oslab/osspracticals$ gcc PROD_CONS.c -o PROD_CONS
tarun@LAPTOP-BRD2JBFC:~/oslab/osspracticals$ ./PROD_CONS
Child: Consumed 1000 messages
Parent: Produced 1000 messages
Communication time: 0.000957 seconds
Efficiency: 1044932.08 messages/second
tarun@LAPTOP-BRD2JBFC:~/oslab/osspracticals$
```

Grep.c

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <unistd.h>
```

```
#include <sys/wait.h>
```

```
int main()
```

```
{
```

```
    int fd[2];
```

```
    pid_t pid1, pid2;
```

```
    // Create pipe
```

```
    if (pipe(fd) == -1)
```

```
    {
```

```
        perror("pipe failed");
```

```
        return 1;
```

```
    }
```

```
    // First child for: ls -l
```

```
    pid1 = fork();
```

```
    if (pid1 < 0)
```

```
    {
```

```
        perror("fork failed");
```

```
    return 1;
}

if (pid1 == 0)
{
    // Redirect standard output to pipe
    dup2(fd[1], STDOUT_FILENO);

    close(fd[0]);
    close(fd[1]);

    // Execute ls -l
    execlp("ls", "ls", "-l", NULL);

    perror("execlp failed");
    exit(1);
}

// Second child for: grep ".c"
pid2 = fork();

if (pid2 < 0)
{
    perror("fork failed");
    return 1;
}

if (pid2 == 0)
```

```

{
    // Redirect standard input from pipe
    dup2(fd[0], STDIN_FILENO);

    close(fd[0]);

    close(fd[1]);

    // Execute grep ".c"
    execlp("grep", "grep", ".c", NULL);

    perror("execlp failed");
    exit(1);
}

// Parent closes both pipe ends
close(fd[0]);
close(fd[1]);

// Wait for both children
wait(NULL);
wait(NULL);

return 0;
}

```

```

tarun@LAPTOP-BRD2JBFC:~/oslab/ossdp/practical5$ gedit pipe_ls.c
^C
tarun@LAPTOP-BRD2JBFC:~/oslab/ossdp/practical5$ gcc pipe_ls.c -o pipe_ls
tarun@LAPTOP-BRD2JBFC:~/oslab/ossdp/practical5$ ./pipe_ls
-rwxr-xr-x 1 sricharan sricharan 16376 Aug 22 03:49 PROD_CONS
-rw-r--r-- 1 sricharan sricharan 1521 Aug 22 03:49 PROD_CONS.c
-rwxr-xr-x 1 sricharan sricharan 16304 Aug 22 03:56 pipe_ls
-rw-r--r-- 1 sricharan sricharan 1211 Aug 22 03:56 pipe_ls.c
tarun@LAPTOP-BRD2JBFC:~/oslab/ossdp/practical5$

```